Teoria Współbieżności

Kajetan Frątczak 01.11.2025 r.

Laboratorium 04

Java Concurrency Utilities

1. Treść zadań

Zadanie 1

Producenci i konsumenci z losowa ilością pobieranych i wstawianych porcji.

- Bufor o rozmiarze 2M.
- Jest m producentów i n konsumentów.
- Producent wstawia do bufora losowa liczbę elementów (nie więcej niż M).
- Konsument pobiera losowa liczbę elementów (nie więcej niż M).
- Zaimplementować przy pomocy monitorów Javy oraz mechanizmów Java Concurrency Utilities.
- Przeprowadzić porównanie wydajności (np. czas wykonywania) vs. różne parametry, zrobić wykresy i je skomentować.

2. Argumentacja rozwiązania

Zadanie 1 (monitor)

Rozwiązanie tego zadania opiera się na monitorze Javy z metodami synchronized oraz mechanizmach wait() i notify(). Bufor przechowuje tablicę elementów, zmienną odpowiedzialną za wielkość bufora oraz licznik zawierający aktualną liczbę elementów w buforze (bufor sam w sobie jest reprezentowany jako tablica o stałym rozmiarze - 2M). Ponadto posiada np. licznik całkowitej liczby elementów do wyprodukowania (totalToProduce). Producent czeka, gdy brakuje miejsca w buforze, a konsument czeka, gdy brakuje elementów do pobrania. Bezpieczeństwo: zostało osiągnięte poprzez użycie słowa kluczowego synchronized na metodach put i get, co zapewnia, że tylko jeden wątek może w danej chwili wykonywać operację na buforze. Mechanizm notifyAll() budzi wszystkie czekające wątki, co zapewnia poprawne działanie przy wielu producentach i konsumentach.

Zadanie 1 (Java Concurrency Utilities)

To rozwiązanie z kolei opiera się na Java Concurrency Utilities (JCU) - wykorzystuje zaawansowane klasy z pakietu java.util.concurrent służące do automatyzacji wielu aspektów synchronizacji. Mechanizmy wykorzystane w kodzie: ArrayBlockingQueue - automatyczne blokowanie przy operacjach na pełnej/pustej kolejce, AtomicInteger - zapewnia atomowe operacje na licznikach, ReentrantLock - dla złożonej synchronizacji, ExecutorService - zarządzanie pulą wątków, CountDownLatch - synchronizuje zakończenie pracy wątków. Bufor jest więc automatyczną kolejka blokującą. Taka implementacja jest bardziej złożona względem tej korzystającej z monitora Javy, jednak osiąga lepszą wydajność przy większej liczbie wątków i ma mniejsze ryzyko błędów synchronizacji.

3. Rozwiązanie

Zadanie 1 (monitor)

```java
package org;

import java.util.Random;

class Producer extends Thread {
   private Buffer _buf;
   private int producerId;
   private int M;
   private Random rand;

   public Producer(Buffer _buf, int producerId, int M ) {
       this._buf = _buf;
       this.producerId =producerId;
       this.M = M;
       this.rand = new Random();
   }

   public void run() {
       while (true) {
           int random_size = rand.nextInt(M) + 1;
           boolean success = _buf.put(random_size, producerId);
           if(!success) break; // produkcja zakończona
           try {
               Thread.sleep(rand.nextInt(200));
           } catch (InterruptedException e) {
               Thread.currentThread().interrupt();
               break;
           }
       }
   }
}

class Consumer extends Thread {
   private Buffer _buf;
   private int consumerId;
   private int M;
   private Random rand;

   public Consumer(Buffer _buf, int consumerId, int M) {
       this._buf = _buf;
       this.consumerId = consumerId;
       this.M = M;
       this.rand = new Random();
   }

   public void run() {
       while (true) {
           int random_size = rand.nextInt(M) + 1;
           boolean success = _buf.get(random_size, consumerId);
           if(!success) break; // konsumpcja zakończona
           try {
               Thread.sleep(rand.nextInt(200));
           } catch (InterruptedException e) {
               Thread.currentThread().interrupt();
               break;
           }
       }
   }
}
```

```java
class Buffer {
   private int[] items;
   private int items_now = 0; // aktualna liczba elementów w buforze
   private int size;
   private int totalToProduce;
   private int produced = 0;
   private int consumed = 0;

   Buffer(int size, int totalToProduce) {
       this.size = size;
       this.items = new int[size];
       this.totalToProduce = totalToProduce;
   }

   // synchronizacja przy pomocy wait() oraz notify()
   public synchronized boolean put(int quantity, int producerId) {
       if (produced >= totalToProduce) return false; // wszystko już wyprodukowano - zakończ

       // przycięcie, jeśli brakuje miejsca do końca całkowitej produkcji
       if (produced + quantity > totalToProduce) {
           quantity = totalToProduce -  produced;
       }

       while (items_now + quantity > size) {
           try {
               wait(); // czekaj jeśli bufor jest pełny
           } catch (InterruptedException e) {
               Thread.currentThread().interrupt();
               return false;
           }
       }
       for (int i = 0; i < quantity; i++) {
           items[items_now + i] = producerId * 1000 + i;
       }
       items_now += quantity;
       produced += quantity;

       System.out.println("Producer " + producerId + " put: " + quantity + " items (buffer: " + items_now +
"/" + size + ")");
       notifyAll();
       return produced < totalToProduce;
   }

   public synchronized boolean get(int quantity, int consumerId) {
       while (items_now < quantity && produced < totalToProduce) {
           try {
               wait(); // czekaj jeśli bufor nie ma wystarczająco elementów
           } catch (InterruptedException e) {
               Thread.currentThread().interrupt();
               return false;
           }
       }
       // jeśli nic nie ma i już nie będzie to kończymy
       if (items_now == 0 && produced >= totalToProduce) return false;

       if (quantity > items_now) {
           quantity = items_now;
       }

       items_now -= quantity;
       consumed += quantity;

       System.out.println("Consumer " + consumerId + " got: " + quantity + " items (buffer: " + items_now +
"/" + size + ")");
       notifyAll();
       return true;
   }
```

```java
    public int getProduced() {
        return produced;
    }
    public int getConsumed() {
        return consumed;
    }
}

public class ProducerConsumer {
    public static void main(String[] args) {
        // synchronizacja przy pomocy monitora: wait(), notify()
        // m producentów n konsumentów
        // 2M - rozmiar bufora
        // Producent wstawia do bufora losowa liczbę elementów (nie więcej niż M)
        // Konsument pobiera losowa liczbę elementów (nie więcej niż M)
        int M = 100;
        int totalElements = 1000; // całkowita liczba elementów do wyprodukowania
        int m = 3; // liczba producentów
        int n = 2; // liczba konsumentów

        // int totalOperations = 300; // tylko, że nie wiemy ile wstawią i pobiorą elementów (bo to jest
losowe) - i wtedy zrobi się deadlock (zakleszczenie)

        long startTime = System.nanoTime();

        Buffer buffer = new Buffer(2*M, totalElements);

        Producer[] producers = new Producer[m];
        for (int i = 0; i < m; i++) {
            producers[i] = new Producer(buffer, i, M);
            producers[i].start();
        }

        Consumer[] consumers = new Consumer[n];
        for (int i = 0; i < n; i++) {
            consumers[i] = new Consumer(buffer, i, M);
            consumers[i].start();
        }

        try {
            for (int i = 0; i < m; i++) {
                producers[i].join();
            }
            System.out.println("\n>>> All producers finished <<<\n");
            for (int i = 0; i < n; i++) {
                consumers[i].join();
            }
            System.out.println("\n>>> All consumers finished <<<\n");
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }

        long endTime = System.nanoTime();
        long exeTime = (endTime - startTime) / 1_000_000;
        System.out.println("=== Podsumowanie ===");
        System.out.println("Wyprodukowano: " + buffer.getProduced());
        System.out.println("Skonsumowano: " + buffer.getConsumed());
        System.out.println("Czas wykonania: " + exeTime + " ms");

    }
}
```

Zadanie 1 (JCU)

```java
package org;
import java.util.Random;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.locks.ReentrantLock;

// wersja z użyciem Java Concurrency Utilities
class ProducerTask implements Runnable {
   private BufferJCU buffer;
   private int producerId;
   private int M;
   private Random rand;

   public ProducerTask(BufferJCU buffer, int producerId, int M ) {
       this.buffer = buffer;
       this.producerId =producerId;
       this.M = M;
       this.rand = new Random();
   }

   @Override
   public void run() {
       while (true) {
           int random_size = rand.nextInt(M) + 1;
           boolean success = buffer.put(random_size, producerId);
           if(!success) break; // produkcja zakończona
           try {
               Thread.sleep(rand.nextInt(200));
           } catch (InterruptedException e) {
               Thread.currentThread().interrupt();
               break;
           }
       }
       System.out.println("Producer " + producerId + " finished");
   }
}
class ConsumerTask implements Runnable {
   private BufferJCU buffer;
   private int consumerId;
   private int M;
   private Random rand;

   public ConsumerTask(BufferJCU buffer, int consumerId, int M) {
       this.buffer = buffer;
       this.consumerId = consumerId;
       this.M = M;
       this.rand = new Random();
   }

   @Override
   public void run() {
       while (true) {
           int random_size = rand.nextInt(M) + 1;
           boolean success = buffer.get(random_size, consumerId);
           if(!success) break; // konsumpcja zakończona
           try {
               Thread.sleep(rand.nextInt(200));
           } catch (InterruptedException e) {
               Thread.currentThread().interrupt();
               break;
           }
       }
       System.out.println("Consumer " + consumerId + " finished");
   }
}
```

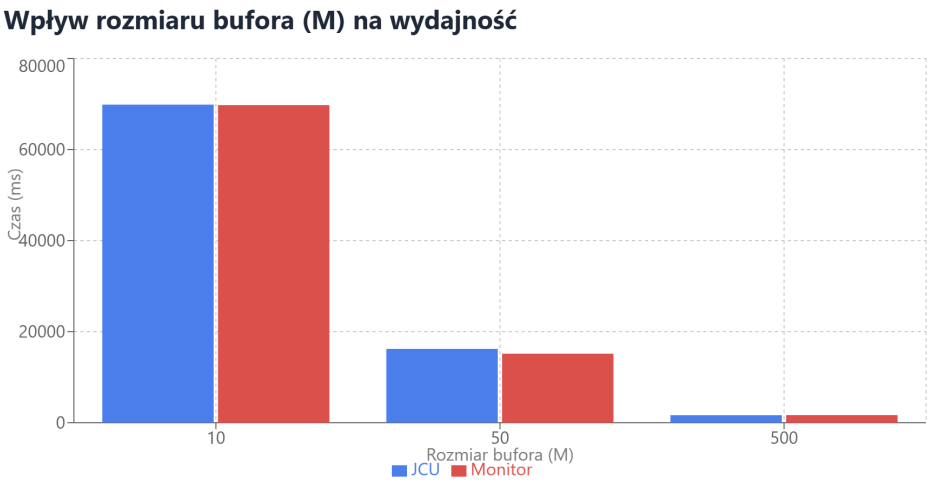
```java
class BufferJCU {
   // BlockingQueue - automaycznie blokuje przy próbie dodania do pełnej kolejki lub pobrania z pustej
kolejki
   private ArrayBlockingQueue<Integer> queue;
   private int size;

   // AtomicInteger - bezpieczne operacje atomowe bez synchronizacji
   private AtomicInteger produced =  new AtomicInteger(0);
   private AtomicInteger consumed =  new AtomicInteger(0);
   private int totalToProduce;

   // ReentrantLock - dla operacji wymagających złożonej synchronizacji
   private ReentrantLock producerLock = new ReentrantLock();
   private ReentrantLock consumerLock = new ReentrantLock();

   private boolean productionFinished = false;

   BufferJCU(int size, int totalToProduce) {
       this.size = size;
       this.totalToProduce = totalToProduce;
       // ograniczona kolejka FIFO
       this.queue = new ArrayBlockingQueue<>(size);
   }

   public boolean put(int quantity, int producerId) {
       producerLock.lock();
       try {
           // sprawdzam, czy trzeba jeszcze coś produkować
           int currentProduced = produced.get();
           if (currentProduced >= totalToProduce) {
               productionFinished = true;
               return false;
           }

           // przycinam jeśli mamy za dużo produkcji
           if (currentProduced + quantity > totalToProduce) {
               quantity = totalToProduce - currentProduced;
           }

           // rezerwuje miejsce w liczniku
           produced.addAndGet(quantity);
       } finally {
           producerLock.unlock();
       }

       try {
           // wstawiam elementy do BlockinQueue
           for (int i = 0; i < quantity; i++) {
               // put() automatycznie blokuje jeśli kolejka jest pełna
               queue.put(producerId * 1000 + i);
           }
           System.out.println("Producer " + producerId + " put: " + quantity + " items (buffer: " +
queue.size() + "/" + size + ", total produced: " + produced.get() + ")");
           return produced.get() < totalToProduce;
       } catch (InterruptedException e) {
           Thread.currentThread().interrupt();
           // cofam rezerwacje w wyniku przerwania
           produced.addAndGet(-quantity);
           return false;
       }
   }

   public boolean get(int quantity, int consumerId) {
       // jeśli produkcja zakończona i kolejka pusta - kończymy
       if  (productionFinished && queue.isEmpty()) {
           return false;
       }
```

```java
        consumerLock.lock();
        int actualQuantity = quantity;
        try {
            // jeśli produkcja zakończona - pobierz tylko to co jest
            if (productionFinished && queue.size() < quantity) {
                actualQuantity = queue.size();
                if (actualQuantity == 0) return false;
            }
        } finally {
            consumerLock.unlock();
        }

        try {
            // pobieranie elementów z BlockingQueue
            for (int i = 0; i < actualQuantity; i++) {
                // poll() z timeoutem - czeka max 1 sekundę na element
                Integer item = queue.poll(1, TimeUnit.SECONDS);
                if (item == null) {
                    // timeout - sprawdź czy produkcja zakończona
                    if (productionFinished && queue.isEmpty()) {
                        actualQuantity = i; // pobrano tylko i elementów
                        break;
                    }
                }
            }

            consumed.addAndGet(actualQuantity);

            System.out.println("Consumer " + consumerId + " got: " + actualQuantity + " items (buffer: " +
queue.size() + "/" + size + ", total consumed: " + consumed.get() + ")");
            return true;

        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            return false;
        }
    }
    public int getConsumed() {
        return consumed.get();
    }

    public int getProduced() {
        return produced.get();
    }
}

public class ProducerConsumerJCU {
    public static void main(String[] args) throws InterruptedException {
        // synchronizacja przy pomocy Java Concurrency Utilities
        // m producentów n konsumentów
        // 2M - rozmiar bufora
        // Producent wstawia do bufora losowa liczbę elementów (nie więcej niż M)
        // Konsument pobiera losowa liczbę elementów (nie więcej niż M)
        int M = 100;
        int totalElements = 1000; // całkowita liczba elementów do wyprodukowania
        int m = 3; // liczba producentów
        int n = 2; // liczba konsumentów

        // pomiar czasu
        long startTime = System.nanoTime();

        BufferJCU bufferJCU = new BufferJCU(2*M, totalElements);

        // ExecutorService - zarządza pulą wątków
        // Tworzę osobne pule dla producentów i konsumentów
        ExecutorService producerExecutor = Executors.newFixedThreadPool(m);
        ExecutorService consumerExecutor = Executors.newFixedThreadPool(n);
```

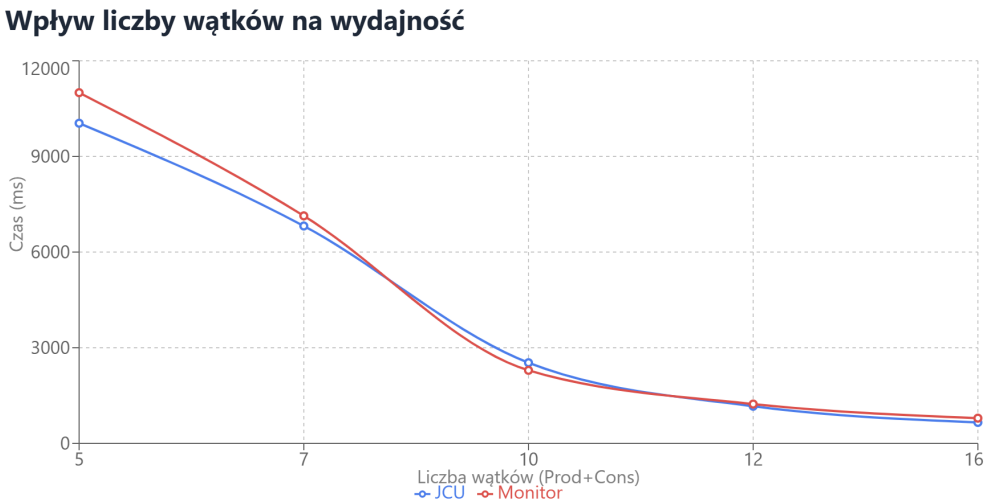
```java
        CountDownLatch producerLatch = new CountDownLatch(m);
        CountDownLatch consumerLatch = new CountDownLatch(n);

        for (int i = 0; i < m; i++) {
            int producerId = i;
            producerExecutor.submit(() -> {
                try {
                    new ProducerTask(bufferJCU, producerId, M).run();
                } finally {
                    producerLatch.countDown();
                }
            });
        }

        for (int i = 0; i < n; i++) {
            int consumerId = i;
            consumerExecutor.submit(() -> {
                try {
                    new ConsumerTask(bufferJCU, consumerId, M).run();
                } finally {
                    consumerLatch.countDown();
                }
            });

        }

        producerExecutor.shutdown();
        consumerExecutor.shutdown();

        producerLatch.await();
        System.out.println("\n>>> All producers finished <<<\n");

        consumerLatch.await();
        System.out.println("\n>>> All consumers finished <<<\n");

        producerExecutor.awaitTermination(5, TimeUnit.SECONDS);
        consumerExecutor.awaitTermination(5, TimeUnit.SECONDS);

        long endTime = System.nanoTime();
        long exeTime  = (endTime - startTime) / 1_000_000;


        System.out.println("=== Podsumowanie ===");
        System.out.println("Wyprodukowano: " + bufferJCU.getProduced());
        System.out.println("Skonsumowano: " + bufferJCU.getConsumed());
        System.out.println("Czas wykonania: " + exeTime + " ms");
    }
}
```

4. Wykresy, tabele, wyniki

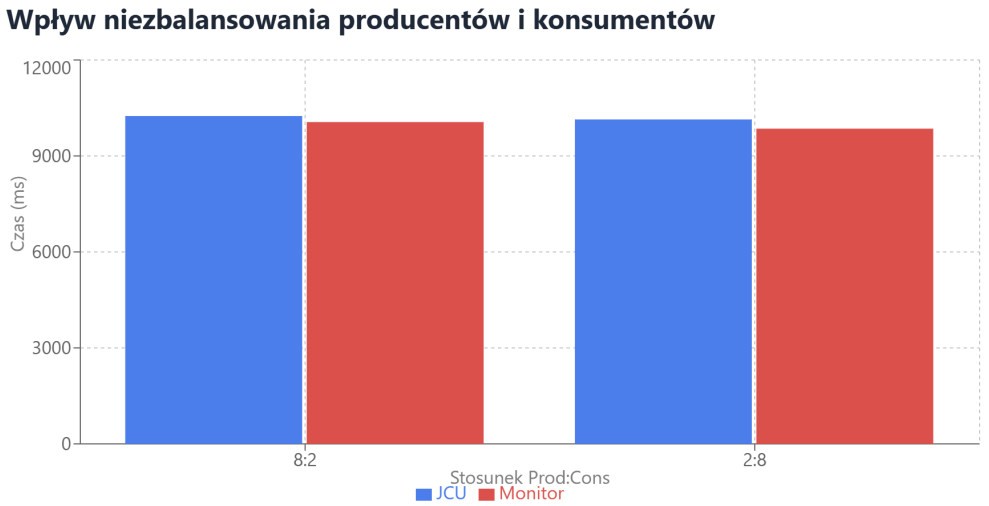
Nr	Typ testu	M	totalElements	Prod.	Kons.	JCU czas (ms)	Monitor czas (ms)
1	Skalowanie	100	10000	3	2	10041	10999
2	Skalowanie	100	10000	4	3	6818	7136
3	Skalowanie	200	10000	5	5	2530	2291
4	Skalowanie	300	10000	6	6	1165	1234
5	Skalowanie	400	10000	8	8	657	791
6	Bufor	10	10000	3	3	69863	69755
7	Bufor	50	10000	3	3	16210	15158
8	Bufor	500	10000	3	3	1651	1668
9	Niezbalansowany	100	10000	8	2	10248	10059
10	Niezbalansowany	100	10000	2	8	10141	9853
11	Porcje	10	10000	4	4	51035	51620
12	Porcje	500	10000	4	4	1196	1197
13	Stress	10	20000	10	10	40049	40215
14	Stress	200	50000	12	12	4472	4570



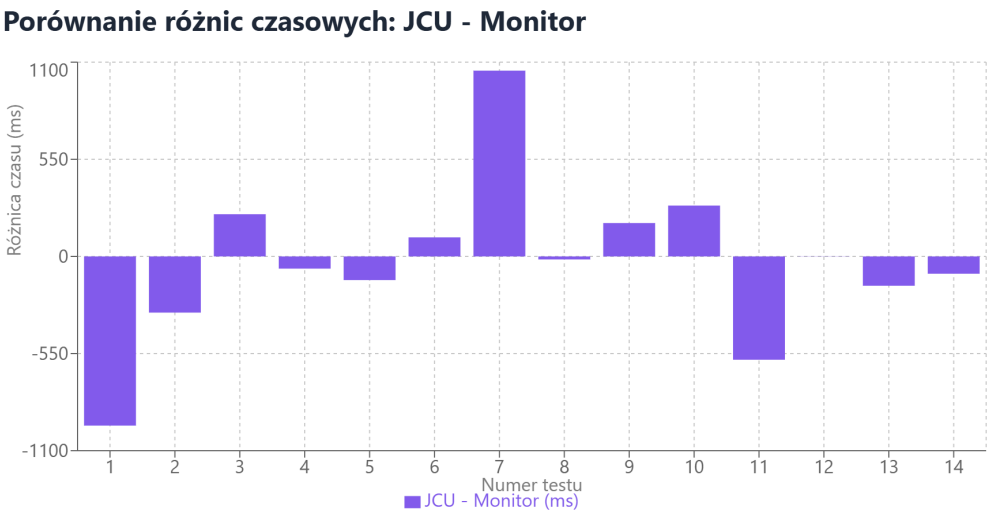
Rysunek 1: Wykres wpływu rozmiaru bufora na wydajność



Rysunek 2: Wykres wpływu liczby wątków na wydajność



Rysunek 3: Wykres wpływu niezbalansowania liczby producentów i konsumentów na wydajność



Rysunek 4: Wykres podsumowujący

5. Wnioski

- Prędkość wykonania: JCU wykazuje lepszą wydajność przy większej liczbie wątków, co wynika z bardziej zaawansowanych mechanizmów zarządzania owymi wątkami.
- Wpływ rozmiaru bufora: wraz ze wzrostem rozmiaru bufora wydajność rośnie (nie ma dużej różnicy w wydajności między monitorem a JCU).
- Konfiguracje niezbalansowane: gdy liczba producentów i konsumentów jest niezrównoważona, oba rozwiązania zachowują się podobnie.
- Podsumowując, JCU w większości testów miało nieznaczną przewagę nad monitorem, szczególnie w scenariuszach z większą ilością wątków (przez lepsze zarządzanie pulą wątków). Jednak przy względnie małych danych wejściowych (a na takich testowałem, żeby nie czekać przysłowiowe pół dnia na wyniki) ta przewaga jest nieznaczna, a monitory są prostsze w implementacji. Wybór rozwiązania zależy więc od celu naszego projektu - czy stawiamy na prostotę czy może na maksymalizację wydajności i zaawansowane funkcje.

6. Bibliografia

http://java.sun.com/j2se/1.5.0/docs/guide/concurrency/overview.html
https://docs.oracle.com/en/java/javase/24/docs/api/java.base/java/util/concurrent/package-summary.html